

Sample Questions: Solutions

June 1, 2026

Problem 1. *DFA Design*

Design a DFA over the alphabet $\Sigma = \{0, 1\}$ that recognizes the language

$$L = \{w \in \{0, 1\}^* \mid w \text{ does not contain } 101 \text{ as a substring}\}.$$

Solution.

We construct a DFA that keeps track of how much of the forbidden substring 101 has been seen as a suffix of the input so far.

Let

$$M = (Q, \Sigma, \delta, q_0, F),$$

where

$$Q = \{q_0, q_1, q_2, q_{\text{dead}}\}, \quad \Sigma = \{0, 1\},$$

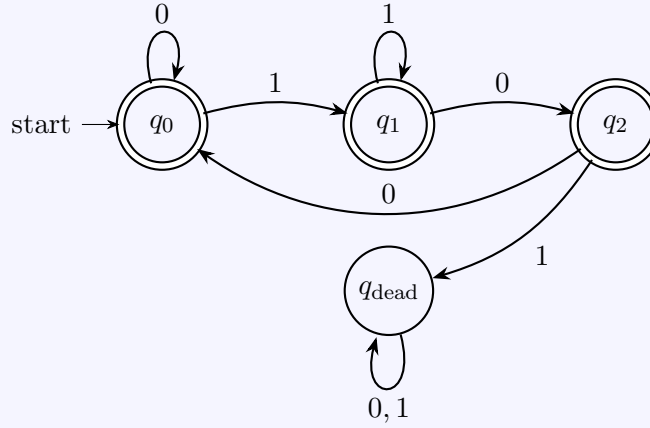
and

$$F = \{q_0, q_1, q_2\}.$$

The states have the following meanings:

- q_0 : we have not just seen any useful prefix of 101,
- q_1 : the current suffix is 1,
- q_2 : the current suffix is 10,
- q_{dead} : we have seen 101 as a substring.

The transition function is given by the following diagram.



The only rejecting state is q_{dead} , which is entered exactly when the substring 101 has appeared. Once this happens, the machine remains in q_{dead} forever (trap state).

Problem 2. Pumping Lemma

Prove that the following language is *not* regular:

$$L = \{0^i 1^j \mid i \leq j, i, j \geq 0\}.$$

Solution.

Assume, for contradiction, that L is regular. Let p be the pumping length given by the pumping lemma.

Choose the string $s = 0^p 1^p$. We have $s \in L$ since $p \leq p$, and $|s| = 2p \geq p$.

By the pumping lemma, $s = xyz$ with:

- $|xy| \leq p$,
- $|y| \geq 1$,
- $xy^k z \in L$ for all $k \geq 0$.

Since $|xy| \leq p$ and s begins with p zeros, the substring y lies entirely within the prefix of 0's. So $y = 0^m$ for some $m \geq 1$.

Pump up: consider $xy^2 z = 0^{p+m} 1^p$. For this to belong to L , we would need $p + m \leq p$, i.e., $m \leq 0$, contradicting $m \geq 1$.

Therefore $xy^2 z \notin L$, contradicting the pumping lemma. Hence L is not regular. $\square$

Problem 3. CFG and Ambiguity

Consider the CFG G of Boolean expressions over $\Sigma = \{\vee, \wedge, (,), p, q, r, s\}$ with rules

$$E \rightarrow E \vee E \mid E \wedge E \mid (E) \mid p \mid q \mid r \mid s.$$

Convention: just as $*$ binds tighter than $+$ in arithmetic, $\wedge$ binds tighter than $\vee$ in Boolean

logic. So $p \vee q \wedge r$ is read as $p \vee (q \wedge r)$ by default.

- (a) Give a leftmost derivation of $p \vee q \wedge r \vee s$ in G .
- (b) Show that G is ambiguous by drawing two distinct parse trees for $p \vee q \wedge r \vee s$.
- (c) Operational significance: with $p = \mathbf{true}$, $q = \mathbf{true}$, $r = \mathbf{false}$, $s = \mathbf{false}$, what Boolean value does each parse tree evaluate to?
- (d) Give an unambiguous CFG G' with $L(G') = L(G)$ that encodes the convention that $\wedge$ binds tighter than $\vee$.

Solution.

- (a) A leftmost derivation matching the standard-convention parse $((p \vee (q \wedge r)) \vee s)$:

$$\begin{aligned}
 E &\Rightarrow E \vee E \\
 &\Rightarrow (E \vee E) \vee E \\
 &\Rightarrow p \vee E \vee E \\
 &\Rightarrow p \vee (E \wedge E) \vee E \\
 &\Rightarrow p \vee q \wedge E \vee E \\
 &\Rightarrow p \vee q \wedge r \vee E \\
 &\Rightarrow p \vee q \wedge r \vee s
 \end{aligned}$$

(The parentheses are shown only to clarify the structure being derived; they are not in the string.)

- (b) Two distinct parse trees, written in bracketed form:

- *Standard parse* ($\wedge$ tighter, $\vee$ left-associative):

$$((p \vee (q \wedge r)) \vee s)$$

Root is $\vee$ with children $(p \vee (q \wedge r))$ and s .

- *Alternative parse* ($\vee$ tighter):

$$((p \vee q) \wedge (r \vee s))$$

Root is $\wedge$ with children $(p \vee q)$ and $(r \vee s)$.

Two different root operators on the same string demonstrates that G is ambiguous.

- (c) With $p = \mathbf{true}$, $q = \mathbf{true}$, $r = \mathbf{false}$, $s = \mathbf{false}$:

- Standard parse: $((p \vee (q \wedge r)) \vee s) = ((T \vee (T \wedge F)) \vee F) = ((T \vee F) \vee F) = T$.
- Alternative parse: $((p \vee q) \wedge (r \vee s)) = ((T \vee T) \wedge (F \vee F)) = (T \wedge F) = F$.

So the two parse trees yield different Boolean values: the choice of grammar literally changes the meaning of the program.

- (d) An unambiguous grammar G' enforcing $\wedge$ tighter than $\vee$:

$$\begin{aligned}
 E &\rightarrow E \vee T \mid T \\
 T &\rightarrow T \wedge F \mid F \\
 F &\rightarrow (E) \mid p \mid q \mid r \mid s
 \end{aligned}$$

Why this works. The three-variable hierarchy E, T, F forces a fixed grouping in every parse tree:

- E generates $\vee$ -expressions whose operands are T 's.
- T generates $\wedge$ -expressions whose operands are F 's.
- F generates atoms or parenthesised expressions.

Because $\wedge$ appears one level deeper in the grammar than $\vee$, every $\wedge$ in a parse tree must occur strictly below any $\vee$ that contains it. So $\wedge$ groups its operands before $\vee$ does. The convention is now baked into the structure of the grammar, not into a reading convention applied after parsing.

Problem 4. PDA Design

Design a pushdown automaton P that recognises the language of marked even-length palindromes over $\{a, b\}$:

$$L = \{ w c w^R \mid w \in \{a, b\}^* \}$$

where $\Sigma = \{a, b, c\}$ and stack alphabet $\Gamma = \{a, b, \$\}$.

- Describe your strategy. What role does the separator c play?
- Give the transitions of P .
- Trace P on $abcba$ and $abcb$.

Solution.

(a) Strategy. The PDA operates in two modes: a push mode (state q_0) where it pushes each input symbol of w onto the stack, and a pop mode (state q_1) where it pops each symbol and matches it against the next input symbol. The separator c is the trigger to switch modes. Without it, the PDA would have to guess nondeterministically where the middle of the string is; with c as a marker, the PDA is deterministic and just transitions on reading c . At the start we push $\$$ as a bottom-of-stack marker so we can detect when the matched portion is fully popped.

(b) Transitions. States: $\{q_{\text{init}}, q_0, q_1, q_{\text{accept}}\}$, start state q_{init} , accept state q_{accept} . We use the notation $x, y \rightarrow z$ (read x , pop y , push z ; ε means none):

q_{init}	$\xrightarrow{\varepsilon, \varepsilon \rightarrow \$}$	q_0	(initialise stack)
q_0	$\xrightarrow{a, \varepsilon \rightarrow a}$	q_0	(push a)
q_0	$\xrightarrow{b, \varepsilon \rightarrow b}$	q_0	(push b)
q_0	$\xrightarrow{c, \varepsilon \rightarrow \varepsilon}$	q_1	(switch modes on c)
q_1	$\xrightarrow{a, a \rightarrow \varepsilon}$	q_1	(match a)
q_1	$\xrightarrow{b, b \rightarrow \varepsilon}$	q_1	(match b)
q_1	$\xrightarrow{\varepsilon, \$ \rightarrow \varepsilon}$	q_{accept}	(accept when stack empty and input done)

- Trace.** We write configurations as (state, remaining input, stack with top on the left).

On *abcba*:

$$\begin{array}{lcl}
(q_{\text{init}}, abcba, \varepsilon) & \xrightarrow{\varepsilon, \varepsilon \rightarrow \$} & (q_0, abcba, \$) \\
& \xrightarrow{a, \varepsilon \rightarrow a} & (q_0, bcba, a\$) \\
& \xrightarrow{b, \varepsilon \rightarrow b} & (q_0, cba, ba\$) \\
& \xrightarrow{c, \varepsilon \rightarrow \varepsilon} & (q_1, ba, ba\$) \\
& \xrightarrow{b, b \rightarrow \varepsilon} & (q_1, a, a\$) \\
& \xrightarrow{a, a \rightarrow \varepsilon} & (q_1, \varepsilon, \$) \\
& \xrightarrow{\varepsilon, \$ \rightarrow \varepsilon} & (q_{\text{accept}}, \varepsilon, \varepsilon)
\end{array}$$

Input is fully consumed and we end in q_{accept} : **accept**.

On *abcb*:

$$\begin{array}{lcl}
(q_{\text{init}}, abcb, \varepsilon) & \xrightarrow{\varepsilon, \varepsilon \rightarrow \$} & (q_0, abcb, \$) \\
& \xrightarrow{a, \varepsilon \rightarrow a} & (q_0, bcb, a\$) \\
& \xrightarrow{b, \varepsilon \rightarrow b} & (q_0, cb, ba\$) \\
& \xrightarrow{c, \varepsilon \rightarrow \varepsilon} & (q_1, b, ba\$) \\
& \xrightarrow{b, b \rightarrow \varepsilon} & (q_1, \varepsilon, a\$)
\end{array}$$

Input is consumed but the stack still has a on top of $\$$. The only ε -transition out of q_1 requires $\$$ on top, which is blocked by the unmatched a . **Reject**.

Problem 5. Decidability

Prove that the complement of every Turing-decidable language is Turing-recognizable.

Solution.

Let L be a Turing-decidable language. By definition, there exists a Turing machine M that decides L . That means that for every input w , the machine M halts and:

$$w \in L \implies M \text{ accepts } w,$$

and

$$w \notin L \implies M \text{ rejects } w.$$

We construct a Turing machine N that recognizes $\bar{L}$, the complement of L .

On input w , the machine N does the following:

1. Run M on input w .
2. If M accepts w , then reject.
3. If M rejects w , then accept.

Since M is a decider, it halts on every input. Therefore, N also halts on every input. Now consider any input w .

If $w \in \bar{L}$, then $w \notin L$. Since M decides L , M rejects w . Therefore, N accepts w .

If $w \notin \bar{L}$, then $w \in L$. Since M decides L , M accepts w . Therefore, N rejects w .

Thus N accepts exactly the strings in $\bar{L}$. Hence $\bar{L}$ is Turing-recognizable.

Therefore, the complement of every Turing-decidable language is Turing-recognizable (also Turing-decidable).

Problem 6. *Complexity Theory*

For each statement below, say whether it is TRUE or FALSE. Briefly justify your answer.

- (a) If A and B are two **NP-complete** languages, then $A \leq_p B$ and $B \leq_p A$.
- (b) If A is an **NP-hard** language, then $A \notin \mathbf{NP}$.
- (c) The set of **NP-complete** languages would change if one measures time complexity using multi-tape deterministic TMs instead of single-tape deterministic TMs.
- (d) If $\mathbf{P} = \mathbf{NP}$, then there exists a deterministic TM that decides the Halting Problem in polynomial time.

Solution.

- (a) **TRUE.**

Since A and B are NP-complete, both are in **NP** and both are NP-hard. Because B is NP-hard, every language in **NP** polynomial-time reduces to B . Since $A \in \mathbf{NP}$, we have

$$A \leq_p B.$$

Similarly, since A is NP-hard and $B \in \mathbf{NP}$, we have

$$B \leq_p A.$$

- (b) **FALSE.**

An NP-hard language is not necessarily outside of **NP**. For example, SAT is NP-complete, so it is NP-hard, but $\text{SAT} \in \mathbf{NP}$. Thus an NP-hard language may or may not be in **NP**.

- (c) **FALSE.**

The set of NP-complete languages would not change. Single-tape and multi-tape deterministic Turing machines can simulate each other with only polynomial overhead. Therefore, the class of polynomial-time computable reductions is the same under either model, so the notion of NP-completeness does not change.

- (d) **FALSE.**

Even if $\mathbf{P} = \mathbf{NP}$, this only concerns languages that are decidable and in $\mathbf{NP}$. The Halting Problem is undecidable, so no deterministic Turing machine decides it at all, let alone in polynomial time.